

Informatika 3

Prednáška 6: Hodnotová sémantika

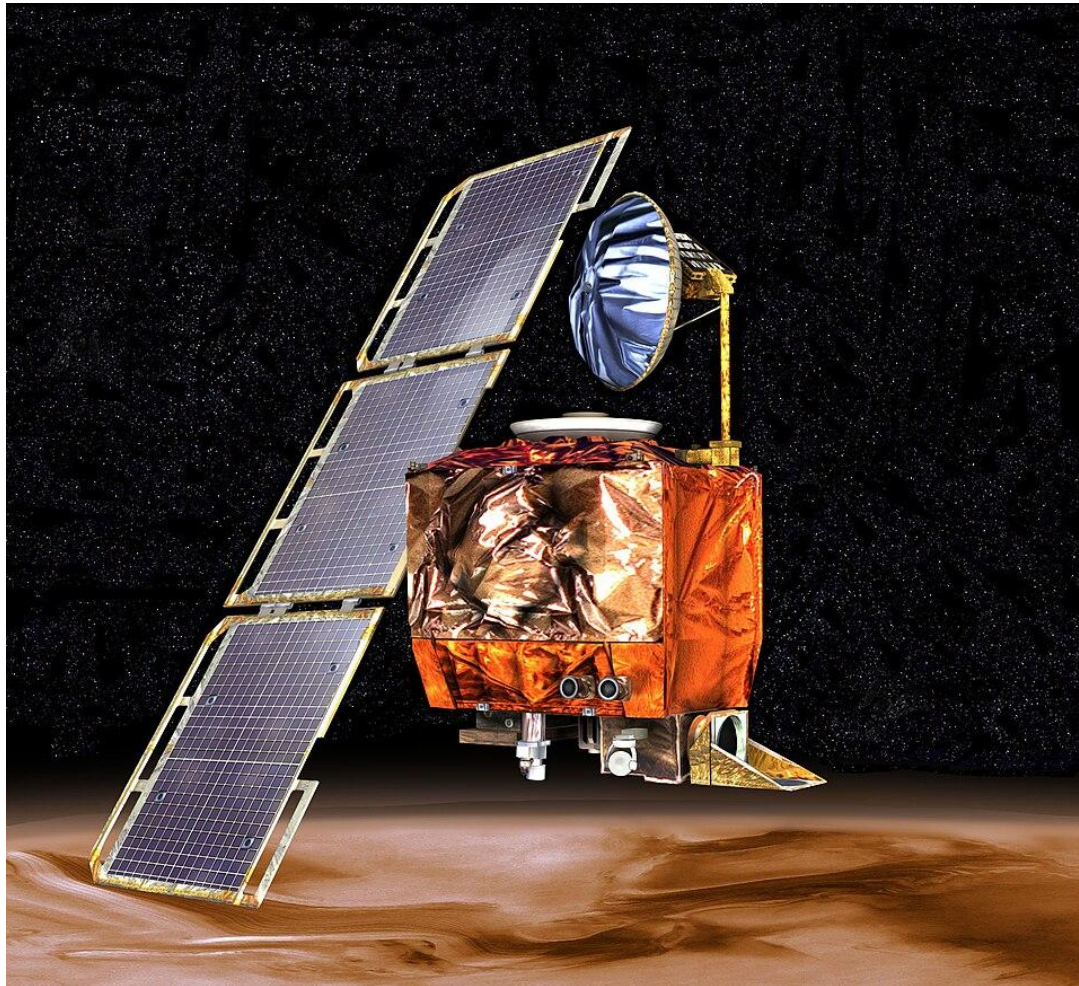
Michal Mrena



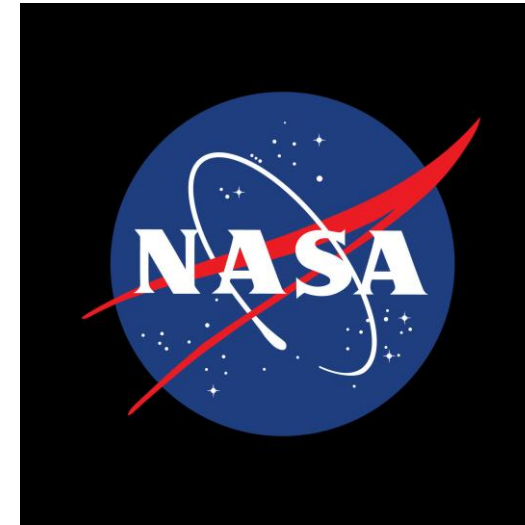
ŽILinskÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Jednoduché triedy





Mars Climate Orbiter



Silne typované čísla

- Na zabezpečenie súladu jednotiek môžeme každú jednotku reprezentovať ako samostatný *silný* typ
- Funkcie môžu namiesto obyčajných čísel (`float`, `int`, ...) preberať inštancie silne typovaných jednotiek

```
Meters calculateElevation(Degrees approach);
```

```
class Degrees {  
public:  
    explicit Degrees(float value);  
    float getValue() const;  
  
private:  
    float m_value;  
};
```

```
class Meters {  
public:  
    explicit Meters(float value);  
    float getValue() const;  
  
private:  
    float m_value;  
};
```

Agregovaný typ

- Jednoduché typy, ktoré reprezentujú údaje bez správania, často reprezentujeme ako štruktúry (`struct`)
- V iných programovacích jazykoch sú takéto entity známe ako záznamy (angl. record)
- V jazyku C++ hovoríme o agregovaných typoch (angl. aggregate type)

```
struct Degrees {  
    float value;  
};  
  
struct Meters {  
    float value;  
};
```

```
struct Vec2 {  
    float x;  
    float y;  
};
```

Agregovaný typ

- Za agregovaný typ považujeme pole alebo triedu, ktorá (od C++20):
 - nemá programátorom deklarovaný konštruktor
 - súkromné alebo chránené členské premenné
 - virtuálneho predka
 - virtuálne členské funkcie

Inicializácia agregovaných typov

- Na inicializáciu agregovaných typov môžeme použiť špeciálnu syntax:

```
Degrees d{180};  
Meters m{2.5};  
Vec2 pos{2.71, 3.14};
```

```
Degrees d = {180};  
Meters m = {2.5};  
Vec2 pos {2.71, 3.14};
```

- od C++20 je možné pomenovať inicializovaných členov:

```
Degrees d{.value = 180};  
Meters m{.value = 2.5};  
Vec2 pos{.x = 2.71, .y = 3.14};
```

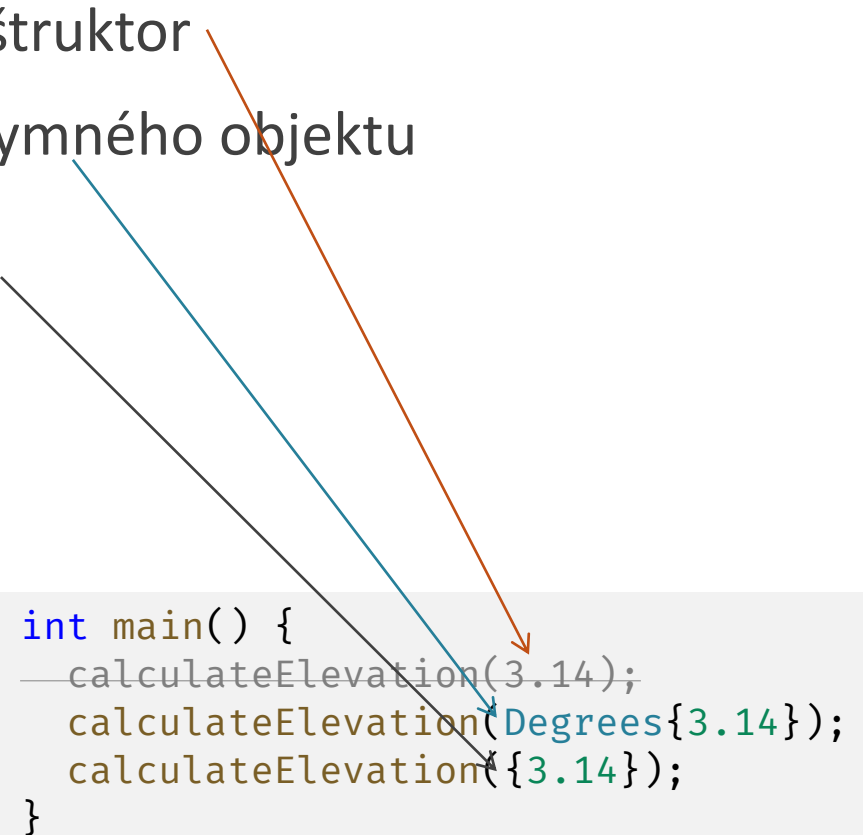
```
Degrees d = {.value = 180};  
Meters m = {.value = 2.5};  
Vec2 pos = {.x = 2.71, .y = 3.14};
```

Inicializácia agregovaných typov

- Agregovaný typ nemá implicitný konverzný konštruktor
- Inicializáciu môžeme použiť na vytvorenie anonymného objektu
- Pri inicializácii parametra môžeme vynechať typ
 - Táto možnosť je však menej explicitná, preto ju v kritickom kóde neodporúčame

```
Meters calculateElevation(Degrees approach);
```

```
int main() {  
    calculateElevation(3.14);  
    calculateElevation(Degrees{3.14});  
    calculateElevation({3.14});  
}
```



Jednotná inicializácia

- Syntax inicializácie používajúcej { } je okrem inicializácie agregovaných typov možné použiť v rôznych situáciách:

```
class Degrees {  
public:  
    explicit Degrees(float value);  
    float getValue() const;  
  
private:  
    float m_value{0};  
};
```

Nastavenie predvolenej hodnoty členskej premennej

```
Degrees::Degrees(float value) :  
    m_value{value}  
{  
}
```

Inicializácia členským premenných v konštruktore.

Jednotná inicializácia

```
class Node {  
public:  
    Node(ICommand *command, Cursor *cursor);  
};
```

```
int main() {  
    tp::Node *node = new tp::Node{nullptr, up};  
}
```

Zavolanie parametrického konštruktora

Keďže je syntax inicializácie pomocou { } možné použiť prakticky všade, nazýva sa aj syntax jednotnej inicializácie (angl. uniform initialization).

```
int main() {  
    int x {10};  
}
```

Inicializácia lokálnej premennej

```
int main() {  
    int x {};  
}
```

Pri použití prázdneho páru { } prebehne hodnotová inicializácia. Jej efekt sa líši podľa typu inicializovaného objektu, pre fundamentálne typy a agregované typy prebehne nulová inicializácia.

Preťažovanie operátorov



Preťažovanie operátorov

- Sčítavanie stupňov je matematicky dobre definované
- Operátor + však o nami definovanom type `Degrees` nič nevie
- S hodnotou preto musím pracovať manuálne sprístupnením členskej premennej `value`

```
class Unit {  
public:  
    void turn(Degrees angle);  
  
private:  
    Degrees m_orientation;  
};  
  
void Unit::turn(Degrees angle) {  
    m_orientation.value =  
        m_orientation.value + angle.value;  
}
```

Preťažovanie operátorov

- V jazyku C++ môžeme definovať preťaženie väčšiny operátorov pre argumenty nami definovaných typov
- Používame na to syntax funkcie so špeciálnym názvom
return_type **operator** *op(parameters)*
- Prípustný návratový typ a prípustný počet a typ parametrov sa pre jednotlivé operátory líši – pravidlá nájdeme v dokumentácii a v komunitnej diskusii
- Niektoré operátory musia byť definované ako členské funkcie, pri niektorých je možné definovať ich aj formou voľnej funkcie

Aritmetika stupňov

```
Degrees operator+(Degrees lhs, Degrees rhs) {  
    return {lhs.value + rhs.value};  
}
```

```
void Unit::turn(Degrees angle) {  
    m_orientation = m_orientation + angle;  
}
```

S inštanciami triedy `Degrees` môžeme po definovaní operátora počítať ako s fundamentálnymi typmi

Podobne by sme vedeli definovať všetky ostatné aritmetické operátory

Aritmetika stupňov

```
struct Degrees {  
    float value;  
    Degrees operator-(Degrees rhs) const;  
    Degrees &operator+=(Degrees rhs);  
};
```

```
Degrees Degrees::operator-(Degrees rhs) const {  
    return {value * rhs.value};  
}
```

```
Degrees &Degrees::operator+=(Degrees rhs) {  
    value = value + rhs.value;  
    return *this;  
}
```

Ak binárny operátor preťažujeme ako členskú funkciu je za jej prvý parameter automaticky považovaný objekt `this`.

Preťažovanie operátorov

```
int main() {  
    Degrees d1{30};  
    Degrees d2{60};  
    Degrees d3{d1 + d2};  
}
```

Vo vygenerovaných inštrukciách môžeme vidieť, že preťažený operátor je na pozadí funkcia so špeciálnym názvom.

```
"main":  
    push    rbp  
    mov     rbp, rsp  
    sub     rsp, 16  
    movss   xmm0, DWORD PTR .LC0[rip]  
    movss   DWORD PTR [rbp-4], xmm0  
    movss   xmm0, DWORD PTR .LC1[rip]  
    movss   DWORD PTR [rbp-8], xmm0  
    movss   xmm0, DWORD PTR [rbp-8]  
    mov     eax, DWORD PTR [rbp-4]  
    movaps  xmm1, xmm0  
    movd    xmm0, eax  
    call    "operator+(Degrees, Degrees)"  
    movss   DWORD PTR [rbp-12], xmm0  
    mov     eax, 0  
    leave  
    ret
```


Preťažovanie operátorov

- Knižnice často používajú preťažené operátory na definovanie doménovo špecifických jazykov
- Pri rozhodovaní o preťažení operátora vždy berte do úvahy čitateľnosť

```
cv::Mat N = (cv::Mat_<int>(3,3) <<
    1, 0, 0,
    0, 1, 0,
    0, 0, 1);
```

Knižnica [OpenCV](#) používa preťaženie `operator`, a `operator<<` na inicializáciu matice na štýl literálu.

```
expression = term >> *(
    ('+' >> term) [ _val += _1 ] |
    ('-' >> term) [ _val -= _1 ]
);

term = factor >> *(
    ('*' >> factor) [ _val *= _1 ] |
    ('/' >> factor) [ _val /= _1 ]
);

factor =
    double_ |
    '(' >> expression >> ')';
```

Knižnica [Boost.Spirit](#) používa preťažené operátory na definovanie gramatík.

Preťažovanie funkcií



Preťažovanie funkcií

- V jazyku C++ môžeme definovať viacero (členských) funkcií s rovnakým názvom, ktoré sa líšia v počte alebo type parametrov

```
#include <numbers>

struct Radians {
    float value;
};

Degrees toDegrees(Radians rad) {
    return {rad.value * 180 / std::numbers::pi};
}
```

```
class Unit {
public:
    void turn(Degrees angle);
    void turn(Radians angle);

private:
    Degrees m_orientation;
};
```

```
void Unit::turn(Radians angle) {
    this->turn(toDegrees(angle));
}
```

Unikátne identifikátory

- Kompilátor je schopný rozlíšiť volania funkcií s rovnakým názvom, pretože interne používa manglované identifikátory

```
int main() {  
    Unit u;  
    u.turn(Degrees{30});  
    u.turn(Radians{1});  
}
```

```
"main":  
    push    rbp  
    mov     rbp, rsp  
    sub     rsp, 16  
    mov     edx, DWORD PTR .LC2[rip]  
    lea     rax, [rbp-4]  
    movd    xmm0, edx  
    mov     rdi, rax  
    call    "_ZN4Unit4turnE7Degrees"  
    mov     edx, DWORD PTR .LC3[rip]  
    lea     rax, [rbp-4]  
    movd    xmm0, edx  
    mov     rdi, rax  
    call    "_ZN4Unit4turnE7Radians"  
    mov     eax, 0  
    leave  
    ret
```

Prenositeľnosť identifikátorov

- Pravidlá manglovania identifikátorov nie sú štandardizované
- Objektové súbory vytvorené rozdielnymi kompilátormi preto nemusia byť kompatibilné – linkovanie by zlyhalo s chybou „undefined reference to“
- Napríklad kompilátory g++ a clang++ používajú [Itanium ABI](#), takže na sú na rovnakej platforme kompatibilné
- Kompilátor MSVC má [vlastnú konvenciu](#)
- Prehľad iných kompilátorov nájdeme [tu](#)

Pravidlo troch



Príklad

```
class Player {  
public:  
    explicit Player(int id);  
private:  
    int m_id;  
};
```

```
Player::Player(int id) :  
    m_id(id)  
{  
}
```

```
int main() {  
    Game g1(10);  
    Game g2(g1);  
}
```

```
class Game {  
public:  
    explicit Game(int duration);  
    ~Game();  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

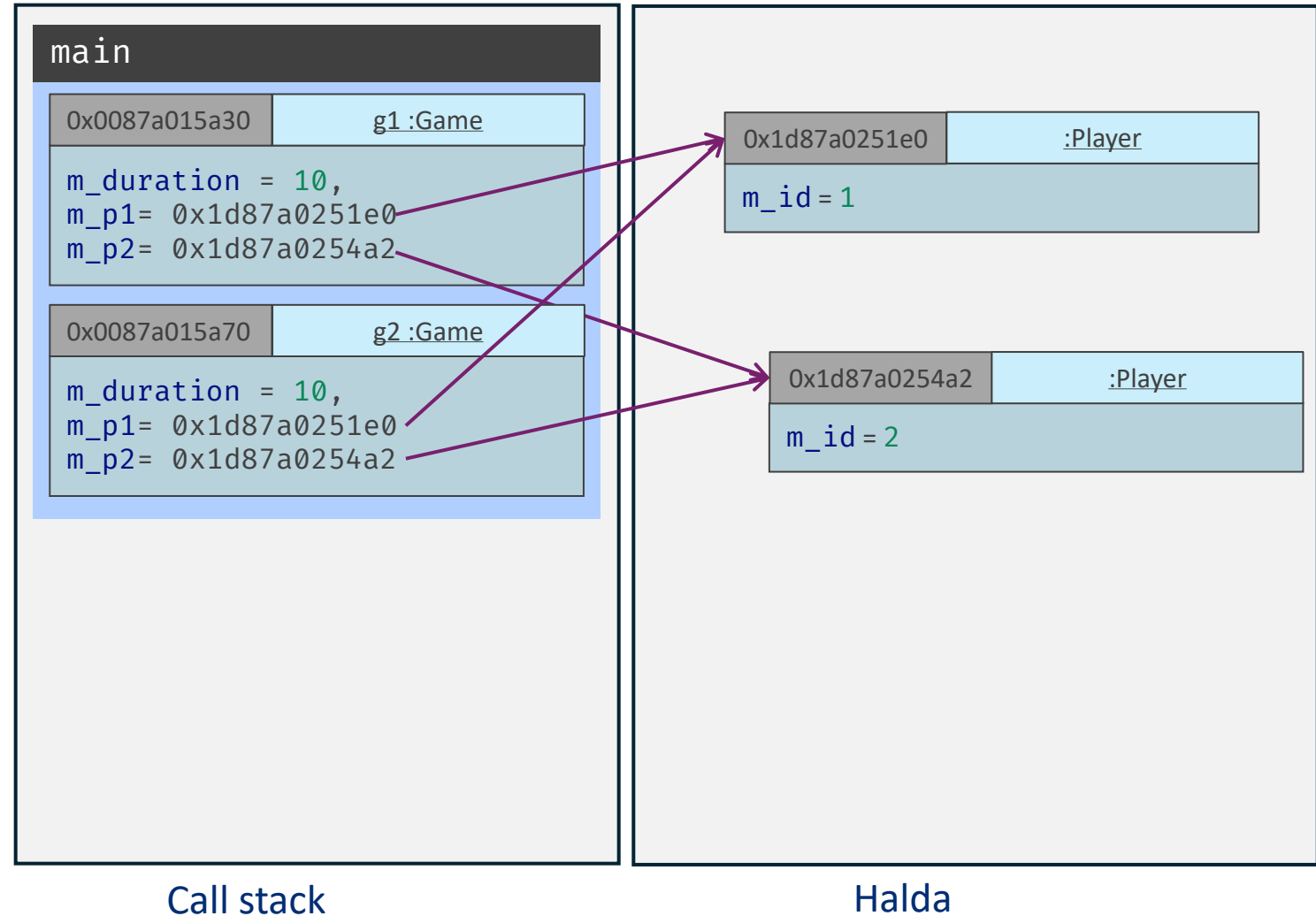
```
Game::Game(int duration) :  
    m_duration(duration),  
    m_p1(new Player(1)),  
    m_p2(new Player(2))  
{  
}
```

```
Game::~~Game() {  
    delete m_p1;  
    delete m_p2;  
}
```

Príklad

```
int main() {  
    Game g1(10);  
    Game g2(g1);  
}
```

Kopírovací konštruktor
vygenerovaný kompilátorom robí iba
plytkú kópiu, čo je v tomto prípade
chyba. Trieda `Game` potrebuje
používateľom definovaný kopírovací
konštruktor, ktorý spraví hlbokú
kópiu.



Príklad

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    ~Game();  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

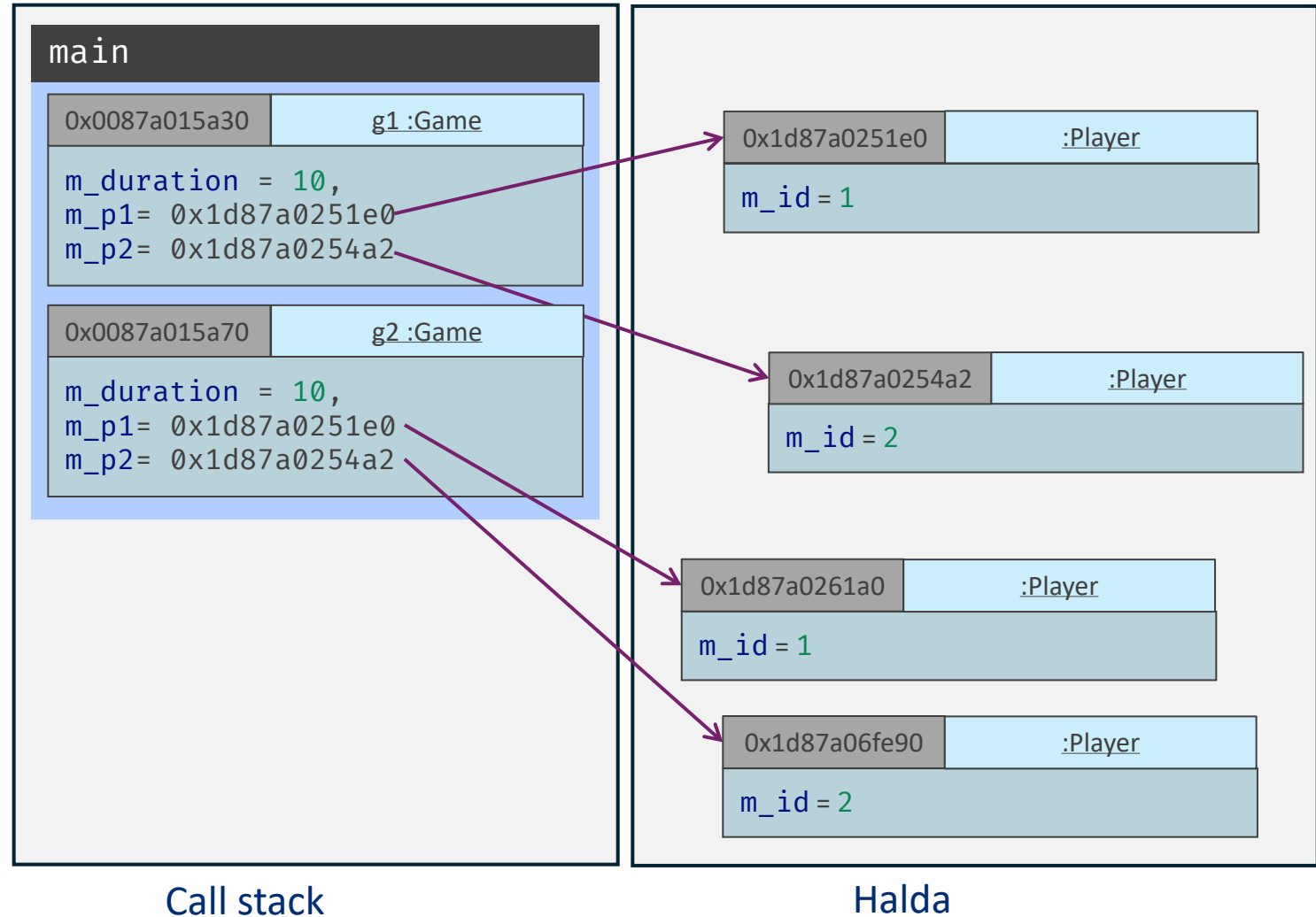
```
Game::Game(const Game &other) :  
    m_duration(other.m_duration),  
    m_p1(new Player(*other.m_p1)),  
    m_p2(new Player(*other.m_p2))  
{  
}
```

Trieda `Player` nevlastní žiadne zdroje, preto je jej kompilátorom vygenerovaný kopírovací konštruktor postačujúci.

Príklad

```
int main() {  
    Game g1(10);  
    Game g2(g1);  
}
```

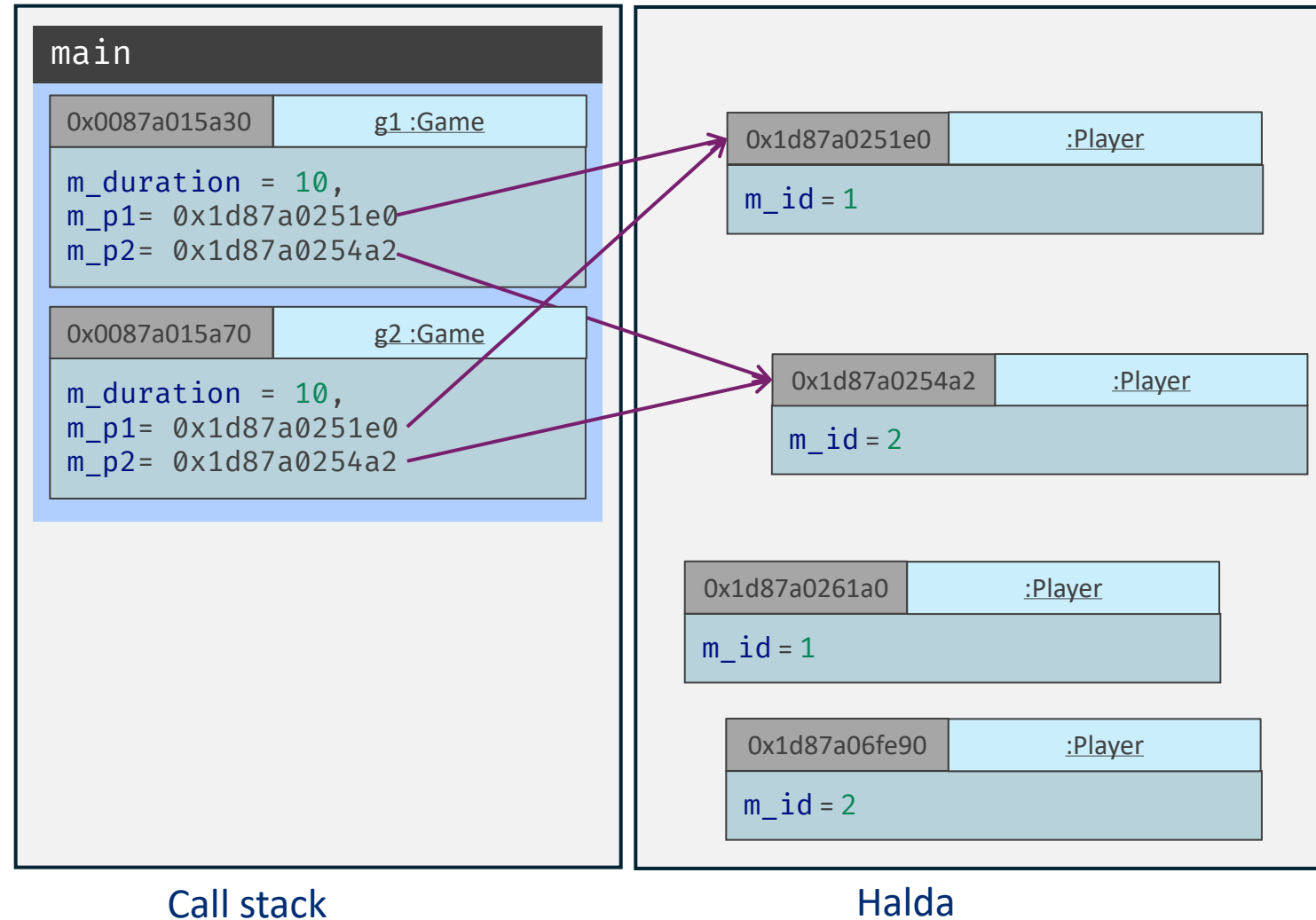
S inštanciami tried môžeme pracovať ako s hodnotami. Hodnoty je možné **priradzovať**.



Príklad

```
int main() {  
    Game g1(10);  
    Game g2(g1);  
    g2 = g1;  
}
```

Za priradenie je zodpovedný **operátor priradenia**. Podobne ako pri kopírovanom konštruktore je v tomto príklade použitý operátor priradenia vygenerovaný kompilátorom, ktorý vytvorí plytkú kópiu.



Operátor priradenia

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    ~Game();  
    Game &operator=(const Game &other);  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

Úlohou operátora priradenia je vytvoriť **hlbokú kópiu** stavu objektu, ktorý sa priradzuje a nastaviť nový stav objektu do ktorého sa priradzuje.

```
Game &Game::operator=(const Game &other) {  
    if (this != &other) {  
        delete m_p1;  
        delete m_p2;  
        m_p1 = new Player(*other.m_p1);  
        m_p2 = new Player(*other.m_p2);  
    }  
    return *this;  
}
```

Operátor priradenia

```
Game &Game::operator=(const Game &other) {  
    if (this != &other) {  
        delete m_p1;  
        delete m_p2;  
        m_p1 = new Player(*other.m_p1);  
        m_p2 = new Player(*other.m_p2);  
    }  
    return *this;  
}
```

Operátor priradenia triedy `T` má presne danú formu:
`T &operator=(const T &other);`

Návratová hodnota je vždy odkaz na objekt, do ktorého sa priradzuje.

V implementácii je vždy potrebné ošetriť prípad samo priradenia (angl. self-assignment).

```
int main() {  
    Game g1(10);  
    Game g2(g1);  
    g1 = g1;  
}
```

Pravidlo troch

- Pravidlo troch hovorí, že ak trieda potrebuje jednu zo špeciálnych metód:
 - Kopírovací konštruktor
 - Deštruktor
 - Operátor priradenia
- je pravdepodobné*, že pre správne fungovanie potrebuje aj zvyšné metódy
- Preto by mala trieda definovať všetky tri súčasne alebo žiadnu z nich

Pochopiteľne, aj toto pravidlo môže mať výnimky. *

Kategórie hodnôt



Kategórie hodnôt

- Každý výraz v jazyku C++ je charakterizovaný svojim **typom** a kategóriou hodnoty
- Poznáme tri primárne kategórie hodnôt:
 - **lvalue** – názov premennej, môžeme získať adresu operátorom addressof
 - **rvalue** – literál, výsledok aritmetickej operácie, volanie funkcie, ktorá vracia hodnotu (nie odkaz), nie je možné získať adresu
 - **xvalue** – člen objektu, ktorý je rvalue
- Ďalej poznáme nasledovné zmiešané kategórie hodnôt:
 - **glvalue** – lvalue alebo xvalue
 - **rvalue** – rvalue alebo xvalue

Téma kategórie hodnôt je pomerne zložitá, prezentujeme iba jej zjednodušený prehľad. *

Kategórie hodnôt

```
class Node {  
public:  
    const std::vector<Node *> &getSubnodes() const;  
};
```

```
int main() {  
    Node *node = new Node{nullptr, up};  
    processNodes(node->getSubnodes());  
}
```

lvalue

```
int main() {  
    int x;  
    x = 20;  
}
```

lvalue

prvalue

```
int main() {  
    Unit u;  
    u.turn(Radians{1});  
}
```

lvalue

prvalue

```
int main() {  
    printFloat(Radians{1}.value);  
}
```

prvalue

xvalue

xvalue – “expiring value”, má identitu, ale jeho životný cyklus čoskoro skončí

lvalue – objekt, ktorý má identitu

prvalue – “pure rvalue” anonymný dočasný objekt

Rvalue odkaz



Lvalue odkaz

- Odkaz ako ho poznáme
- Môže odkazovať iba na lvalue

```
int getNumber() {  
    return 4;  
}  
  
int main() {  
    int &x = getNumber();  
}
```

```
degress.cpp:103:23: error: cannot bind non-const lvalue reference of  
type 'int&' to an rvalue of type 'int'  
103 |     int &x = getNumber();  
    |           ~~~~~^~
```

V príklade funkcia `getNumber` vráti dočasnú hodnotu, na ktorú nie je možné odkazovať.

Lvalue odkaz

- Odkaz ako ho poznáme
- Môže odkazovať iba na lvalue alebo rvalue – iba ak ide o odkaz na `const` objekt

```
int getNumber() {  
    return 4;  
}  
  
int main() {  
    const int &x = getNumber();  
}
```

OK

V príklade funkcia `getNumber` vráti dočasnú hodnotu. Životnosť dočasného objektu je predĺžená po dobu platnosti rozsahu odkazu.

Rvalue odkaz

- Nový typ odkazu od C++11
- Odkazuje na **rvalue** hodnoty
- Syntax: <typ> &&
- Príklady:
 - `int &&` – rvalue odkaz na typ `int`
 - `double &&` – rvalue odkaz na typ `double`
 - `Node&&` – rvalue odkaz na typ `Node`

Rvalue odkaz

```
1 void runGame(Game &g) {  
2     // ...  
3 }  
4  
5 void runGame(Game &&g) {  
6     // ...  
7 }  
8  
9 int main() {  
10     Game g(10);  
11     runGame(g); // zavolá (1)  
12     runGame(Game(20)); // zavolá (5)  
13 }
```

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    ~Game();  
    Game &operator=(const Game &other);  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

Rvalue odkaz

```
1 void runGame(Game &g) {
2     // ...
3 }
4
5 void runGame(Game &&g) {
6     // ...
7 }
8
9 void runGame(Game g) {
10    // ...
11 }
12
13 int main() {
14     Game g(10);
15     runGame(g);
16     runGame(Game(20));
17 }
```

```
class Game {
public:
    explicit Game(int duration);
    Game(const Game &other);
    ~Game();
    Game &operator=(const Game &other);
private:
    int m_duration;
    Player *m_p1;
    Player *m_p2;
};
```

```
game.cpp:72:12: error: call of overloaded
'runGame(Game&)' is ambiguous
    72 |         runGame(g);
        |         ~~~~~^~
```

Rvalue odkaz

```
1 void runGame(const Game &g) {  
2     // ...  
3 }  
4  
5 void runGame(Game &&g) {  
6     // ...  
7 }  
8  
9 int main() {  
10     Game g(10);  
11     runGame(g); // zavolá (1)  
12     runGame(Game(20)); // zavolá (5)  
13 }
```

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    ~Game();  
    Game &operator=(const Game &other);  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

Tento prípad je zdanlivo tiež nejednoznačný – lvalue odkaz môže odkazovať aj na `const` kvalifikovanú rvalue hodnotu. Ak však existuje funkcia preťažená pre rvalue odkaz, zavolá sa prednostne.

Pretypovanie

- Lvalue odkaz je možné pretypovať na rvalue odkaz s použitím `static_cast`
- Preto, ak máme k dispozícii dve preťaženia funkcie pre lvalue a rvalue odkaz, dokážeme si v mieste volania vybrať

```
1 void runGame(Game &g) {  
2     // ...  
3 }  
4  
5 void runGame(Game &&g) {  
6     // ...  
7 }  
8  
9 int main() {  
10     Game g(10);  
11     runGame(static_cast<Game&&>(g)); // zavolá (5)  
12     runGame(Game(20)); // zavolá (5)  
13 }
```

xvalue



Move sémantika



Move konštruktor

- Rvalue odkaz nachádza využitie najčastejšie pri definícii konštruktorov
- Okrem kopírovacieho konštruktoru poznáme tzv. **move konštruktor**
- Move konštruktor triedy **T** má presne danú formu:
`T(T &&other);`
- Move konštruktor sa zavolá, ak sa pokúšame inštanciu inicializovať xvalue hodnotou rovnakého typu

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    Game(Game &&other) noexcept;  
    ~Game();  
    Game &operator=(const Game &other);  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

Move konštruktor

- Keďže je parameter z kategórie xvalue, vieme, že bude čoskoro zanikať, preto môžeme **prevziať** jeho zdroje (už ich nebude potrebovať)
- **Ušetríme** tak alokovanie nových zdrojov
- „**Moved from**“ objekt (*other*) musí zostať vo *valídnom ale nešpecifikovanom stave*

```
Game::Game(Game &&other) noexcept :  
    m_duration(other.m_duration),  
    m_p1(other.m_p1),  
    m_p2(other.m_p2)  
{  
    other.m_p1 = nullptr;  
    other.m_p2 = nullptr;  
}
```

`noexcept` kvalifikátor funkcie označuje, že vo funkcii nemôže prísť k vyhodneniu výnimky. Pri move konštruktoch je dobré používať ho a zabezpečiť, že platí.

Move

```
1 void logState(const Game &game) {
2     // ...
3 }
4
5 void runGame(Game game) {
6     // ...
7 }
8
9 int main() {
10     Game g(10);
11     logState(g);
12     runGame(g);
13     // ...
14 }
```

```
class Game {
public:
    explicit Game(int duration);
    Game(const Game &other);
    Game(Game &&other) noexcept;
    ~Game();
    Game &operator=(const Game &other);
private:
    int m_duration;
    Player *m_p1;
    Player *m_p2;
};
```

Ktorý z konštruktorov triedy `Game` sa použije na inicializáciu parametra `game` (5) pri volaní funkcie `runGame` (12)?

Move

```
1 void logState(const Game &game) {
2     // ...
3 }
4
5 void runGame(Game game) {
6     // ...
7 }
8
9 int main() {
10     Game g(10);
11     logState(g);
12     runGame(g);
13     // ...
14 }
```

```
class Game {
public:
    explicit Game(int duration);
    Game(const Game &other);
    Game(Game &&other) noexcept;
    ~Game();
    Game &operator=(const Game &other);
private:
    int m_duration;
    Player *m_p1;
    Player *m_p2;
};
```

Použije sa jej **kopírovací konštruktor**, pretože parameter `g` (12) je **lvalue**. Ak by sme však vo funkcii `main` od riadku (13) ďalej už nepotrebovali objekt `g`, išlo by o zbytočnú kópiu. Ako by sme mohli *presvedčiť* kompilátor, aby parameter inicializoval **move konštruktorom**?

Move

```
1 void logState(const Game &game) {
2     // ...
3 }
4
5 void runGame(Game game) {
6     // ...
7 }
8
9 int main() {
10     Game g(10);
11     logState(g);
12     runGame(static_cast<Game&&>(g));
13     // ...
14 }
```

```
class Game {
public:
    explicit Game(int duration);
    Game(const Game &other);
    Game(Game &&other) noexcept;
    ~Game();
    Game &operator=(const Game &other);
private:
    int m_duration;
    Player *m_p1;
    Player *m_p2;
};
```

Použitím pretypovania spravíme z parametra **xvalue** a kompilátor vyberie **move konštruktor**.

Move

```
1 #include <utility>
2
3 void logState(const Game &game) {
4     // ...
5 }
6
7 void runGame(Game game) {
8     // ...
9 }
10
11 int main() {
12     Game g(10);
13     logState(g);
14     runGame(std::move(g));
15     // ...
16 }
```

```
class Game {
public:
    explicit Game(int duration);
    Game(const Game &other);
    Game(Game &&other) noexcept;
    ~Game();
    Game &operator=(const Game &other);
private:
    int m_duration;
    Player *m_p1;
    Player *m_p2;
};
```

Štandardná knižnice v hlavičke `<utility>` definuje funkciu [move](#), ktorá za nás spraví pretypovanie.

Move operátor priradenia

- Podobne ako pri kopírovanom konštruktore poznáme aj move operátor priradenia

```
Game &Game::operator=(Game &&other) noexcept {  
    if (this != &other) {  
        delete m_p1;  
        delete m_p2;  
        m_p1 = other.m_p1;  
        m_p2 = other.m_p2;  
        other.m_p1 = nullptr;  
        other.m_p2 = nullptr;  
    }  
    return *this;  
}
```

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    Game(Game &&other) noexcept;  
    ~Game();  
    Game &operator=(const Game &other);  
    Game &operator=(Game &&other) noexcept;  
  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

Pravidlo piatich

- (od C++11) Pravidlo piatich hovorí, že ak trieda potrebuje jednu zo špeciálnych metód:
 - Kopírovací konštruktor
 - Move konštruktoru
 - Deštruktor
 - Operátor priradenia
 - Move operátor priradenia
- je pravdepodobné*, že pre správne fungovanie potrebuje aj zvyšné metódy
- Preto by mala trieda definovať všetkých päť súčasne alebo žiadnu z nich

Pochopiteľne, aj toto pravidlo môže mať výnimky. *

Pravidlo nuly

- Ak trieda **manuálne** nespravuje žiadne zdroje (nealokuje pamäť, neotvára (nízkoúrovňovo) súbory, ...) je lepšie ak nedefinuje žiadnu z piatich operácií (z pravidla piatich) ale spoľahne sa na kompilátorom vygenerované definície týchto operácií
- Toto pravidlo nazývame pravidlo nuly (angl. rule of zero)

RAII



RAII

- Pravidlo nuly môže byť *zdanlivo* ťažké použiť, ak potrebujeme manuálne pracovať so zdrojmi ako pamäť a súbory
- Hodnotová sémantika jazyka C++ nám však značne uľahčuje situáciu
- Kľúčom je využitie návrhového vzoru RAII (angl. Resource Acquisition Is Initialization)
- Kľúčovou myšlienkou je previazanie **životného cyklu zdroja** (napríklad dynamicky alokovanej pamäte) s **životným cyklom objektu** – ktorého deštruktor je volaný **automaticky** pri konci jeho rozsahu platnosti

Manuálna správa zdrojov

```
void runGame() {  
    Player *p = new Player(1);  
  
    if (!checkSomething(*p)) {  
        delete p;  
        return;  
    }  
  
    if (!checkSomethingElse(*p)) {  
        delete p;  
        return;  
    }  
  
    doSomething(*p);  
  
    delete p;  
}
```

Pri predčasnom ukončení funkcie nesmieme zabudnúť uvoľniť všetky zdroje.

Ak sa počas vykonávania funkcie vyhodí výnimka, ktorá nie je odchytená, zdroje sa neuvoľnia.

Automatizovanie správy zdrojov

```
class PlayerPtr {  
public:  
    PlayerPtr();  
    explicit PlayerPtr(Player *p);  
    PlayerPtr(const PlayerPtr &other) = delete;  
    PlayerPtr(PlayerPtr &&other) noexcept;  
    ~PlayerPtr();  
    Player *get() const;  
    Player &operator*();  
    PlayerPtr &operator=(const PlayerPtr &other) = delete;  
    PlayerPtr &operator=(PlayerPtr &&other) noexcept;  
  
private:  
    Player *m_ptr;  
};
```

(od C++11) Syntax = `delete` explicitne zakáže kompilátoru vo vygenerovaní danej funkcie.

```

PlayerPtr::PlayerPtr() : m_ptr(nullptr) {
}

PlayerPtr::PlayerPtr(Player *p) : m_ptr(p) {
}

PlayerPtr::PlayerPtr(
    PlayerPtr &&other
) noexcept :
    m_ptr(other.m_ptr)
{
    other.m_ptr = nullptr;
}

PlayerPtr::~~PlayerPtr() {
    delete m_ptr;
}

```

```

Player *PlayerPtr::get() const {
    return m_ptr;
}

Player &PlayerPtr::operator*() {
    return *m_ptr;
}

PlayerPtr &PlayerPtr::operator=(
    PlayerPtr &&other
) noexcept {
    if (this != &other) {
        delete m_ptr;
        m_ptr = other.m_ptr;
        other.m_ptr = nullptr;
    }
    return *this;
}

```


Automatizovanie správy zdrojov

```
void runGame() {  
    PlayerPtr p(new Player(1));  
  
    if (!checkSomething(*p)) {  
        return;  
    }  
  
    if (!checkSomethingElse(*p)) {  
        return;  
    }  
  
    doSomething(*p);  
}
```

Životný cyklus dynamicky alokovanej inštancie triedy `Player` je teraz previazaný so životným cyklom lokálneho objektu typu `PlayerPtr`.

Tento zanikne automaticky po skončení vykonávania funkcie `runGame` (či už štandardne alebo po vyhodení výnimky), pričom vo svojom deštruktore uvoľní alokovanú pamäť.

unique_ptr

- Manuálna tvorba tried ako `PlayerPtr` by bola *neúnosná*
- Štandardná knižnica preto poskytuje šablónu triedy `unique_ptr` v hlavičke `<memory>`
- Poskytuje rozšírenú funkcionálnu našej triedy `PlayerPtr`

```
#include <memory>

void runDifferentGame() {
    std::unique_ptr<Player> p(new Player(1));

    if (!checkSomething(*p)) {
        return;
    }

    if (!checkSomethingElse(*p)) {
        return;
    }

    doSomething(*p);
}
```

unique_ptr

- Typicky ju používame s *továrenskou* šablónou funkcie make_unique
- Parametrami funkcie sú parametre konšuktora triedy, ktorej inštanciu chceme vytvoriť
- Typ triedy posielame ako *šablónový parameter*
- Jej použitie je bezpečnejšie ako priame volanie operátora `new`

```
#include <memory>

void runDifferentGame() {
    std::unique_ptr<Player> p =
        std::make_unique<Player>(1);

    if (!checkSomething(*p)) {
        return;
    }

    if (!checkSomethingElse(*p)) {
        return;
    }

    doSomething(*p);
}
```

Zhrnutie

- Agregované typy je vhodné použiť pri reprezentácii **hodnôt** bez správania
- Preťažovanie operátorov je dôležitou súčasťou jazyka
 - Pri definovaní vlastných preťažených operátorov je dôležité dbať na **čitateľnosť** a **intuitívnosť** použitia
- Je dôležité vedieť rozlišovať medzi rvalue a lvalue odkazmi
- Move sémantika je dôležitou súčasťou jazyka, ktorej je potrebné rozumieť
- Správu zdrojov (pamäte) je možné automatizovať použitím návrhového vzoru RAI

Ďakujem za pozornosť

Priestor na otázky

